catch ambiguity before it gets cemented into shipped code, according to Mesaros, who explains with a relatable example. "A requirement may say, as a player, when I click a button, a thing will be deleted. The question is how it will be deleted. Will it be soft deleted or hard deleted? Those are some of the big questions that can actually bring the most expensive bugs — even before a first line of code is written."

Kiro layers additional checks on top of its verification steps, using neurosymbolic AI hand in hand with the language model to query requirements, in tandem with additional tests generated from the design doc. As Mesaros put it, "it's so much more than just an LLM writing a markdown file and reading it again." His own reason for wanting it is less architectural, "I hate when all of a sudden an agent is producing thousands of lines of code that I have no idea what's going on."

Vibe coding isn't the enemy

If you expected the spec evangelist to tell you to stop vibe coding, he won't. Mesaros believes vibe coding is here to stay, and increasingly a necessary step in how code is generated going forward.

"I think vibe coding is a viable option," he said. "Vibe coding is an excellent way to quickly get stuff out. Just prototypes and quick turnaround is just fantastic, and I think people should be exploring vibe coding as much as they can. But at a certain point, when you start introducing potential sensitive information, security, you want to make sure those things are actually quite solid. So the existence of spec-driven development doesn't mean you should stop vibe coding."

It all depends on your code's blast radius, as developers like to call it – how your code will impact real world scenarios. Vibe coding is great for fast prototyping, but should be scrutinised extra carefully whenever it touches credentials, money or users.

"While spec-driven development slows you down initially, you're actually done faster ultimately, and it potentially saves tokens and money," Mesaros said. "When I'm certain on the spec, it can just literally one-shot things."

Developer need to control agents and AI slop

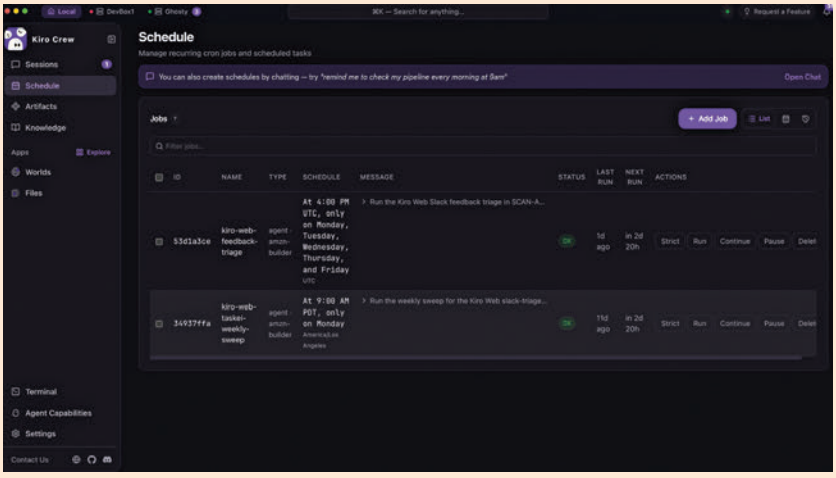
Ultimately, it all comes down to how good is the human in the loop, according to Mesaros. "A developer who knows what they're doing plus an agent is so much better than a developer who doesn't know what they're doing plus an agent."

He explains it with logic that's hard to argue against. "An agent is pretty good at building code from scratch, but you'll probably run into problems. What about operational metrics? How do you maintain it for the long term? So I always encourage my fellow developers to still keep honing the developer craft."

Mesaros redefines the job description of a modern day developer in the AI era. "The role of the developer isn't the person who types in the actual code, per se, but the person who orchestrates different agents to produce high-quality code."

Kiro Crew: Factsheet

- **What it is:** Kiro Crew's an open-source agentic workspace launched to turn AI agents into autonomous, self-learning teammates that maintain context across sessions.
- **Core features:** It coordinates multiple agents, runs scheduled and long-running tasks, and incorporates a seven-layer defense-in-depth security model.
- **Origin:** Kiro began as an internal side project by three Amazon engineers to help them code better, expanding to over 39,000 Amazon builders within six months.
- **Developer engagement:** It has garnered over 30,000 downloads within its first week and surpassed 1,000 GitHub stars within 24 hours of its open-source announcement.
- **India Impact:** Kiro Crew empowered Indian companies like Dhan to ship a complex real-time charting platform with a single engineer in 8 weeks instead of 12 to 24 months.



Read more on Digit.in:
tinyurl.com/3azvf7r7